
AppLock Pro

AI-Powered Privacy & App Locker for macOS

Complete Technical Report

AppLockPro Project

July 6, 2026

Project Name:	FaceLock Pro (AppLockPro)
Version:	1.0.0
Platform:	macOS 14.0+ (Sonoma)
Target Hardware:	Apple Silicon (M1, M2, M3, M4)
Language:	Swift 5.9+
Architecture:	MVVM (Model-View-ViewModel)
AI Processing:	100% Offline

Contents

1	Introduction	5
1.1	Project Overview	5
1.2	Problem Statement	5
1.3	Project Objectives	5
1.4	Scope	6
1.4.1	In Scope	6
1.4.2	Out of Scope	6
2	Core Features	7
2.1	AI Face Unlock	7
2.1.1	Technical Details	7
2.1.2	Cosine Similarity	7
2.2	Application Locker	8
2.2.1	App Discovery	8
2.2.2	Data Model	8
2.3	Background Protection	8
2.3.1	Monitoring Mechanism	8
2.3.2	Interception Flow	9
2.3.3	macOS API Constraints	9
2.4	Face Enrollment Wizard	9
2.4.1	Process	9
2.5	Liveness Detection	10
2.5.1	Tier 1: Vision Framework (No Additional Model)	10
2.5.2	Tier 2: MiniFASNet (Optional Advanced Model)	11
2.6	Multi-Factor Authentication System	11
2.6.1	Authentication Methods	11
2.6.2	Authentication Cascade	11
2.6.3	Touch ID and macOS Password Integration	12
2.6.4	PIN Authentication	12
2.7	Session Management	12
2.7.1	Session Timeout Options	13
2.7.2	Auto-Lock Triggers	13
2.8	Activity Logging	13
2.8.1	Event Types	13
2.8.2	Log Entry Schema	13
2.9	Notifications	14
2.10	Settings and Configuration	14
3	Technical Architecture	15

3.1	Design Pattern: MVVM	15
3.2	Layered Architecture	15
3.2.1	Layer 1: Presentation Layer (SwiftUI + AppKit)	15
3.2.2	Layer 2: Business Logic Layer (Managers)	16
3.2.3	Layer 3: Machine Learning Layer	16
3.2.4	Layer 4: Storage Layer	16
3.2.5	Layer 5: System Layer	17
3.3	Project File Structure	17
4	AI Pipeline	19
4.1	Pipeline Overview	19
4.2	ArcFace Model	20
4.2.1	Model Specifications	20
4.2.2	Model Conversion Pipeline	20
4.2.3	Preprocessing Requirements	21
4.3	MiniFASNet Anti-Spoofing Model	21
4.4	Hardware Acceleration	21
5	Technology Stack	22
5.1	Complete Technology Table	22
5.2	System Requirements	23
5.3	Build and Run Instructions	23
6	Security Design	24
6.1	Security Principles	24
6.2	Data Protection Matrix	24
6.3	Threat Model	25
6.4	Keychain Integration	25
7	User Interface Design	27
7.1	Design Principles	27
7.2	Application Screens	27
7.2.1	Welcome / Onboarding (4 Steps)	27
7.2.2	Dashboard	27
7.2.3	Protected Apps	28
7.2.4	Face Enrollment Wizard	28
7.2.5	Unlock Dialog	28
7.2.6	Activity History	28
7.2.7	Settings	28
7.2.8	About	28
7.3	Navigation Architecture	29
8	Performance	30
8.1	Performance Targets	30
8.2	Optimization Strategies	30
9	Implementation Roadmap	31
9.1	Milestone 1: Project Setup & UI Shell (Complete)	31
9.2	Milestone 2: Camera & Face Detection	31

9.3	Milestone 3: AI Face Recognition	31
9.4	Milestone 4: App Monitoring & Protection	32
9.5	Milestone 5: Multi-Factor Authentication	32
9.6	Milestone 6: Liveness Detection	32
9.7	Milestone 7: Polish & Release	32
9.8	Estimated Timeline	33
10	Testing Strategy	34
10.1	Unit Tests	34
10.2	Integration Tests	34
10.3	Manual Verification Checklist	34
11	Risk Assessment	36
12	Future Enhancements	37
13	Conclusion	38
14	References	39

Chapter 1

Introduction

1.1 Project Overview

FaceLock Pro is a professional macOS desktop application designed to add an intelligent privacy layer to the operating system. By requiring biometric authentication — specifically facial recognition — before allowing access to user-designated applications, it ensures that sensitive data remains secure even if the Mac is left unlocked.

A critical tenet of this project is privacy: the application runs entirely offline. No biometric or personal data is ever uploaded to external servers. It is optimized exclusively for Apple Silicon Macs (M1, M2, M3, M4) to provide fast, efficient, and battery-friendly authentication by leveraging the Apple Neural Engine for on-device machine learning inference.

1.2 Problem Statement

macOS provides system-level login security (password, Touch ID, Apple Watch unlock), but once a user is logged in, all applications are freely accessible. This creates a privacy gap in several scenarios:

- A user steps away from an unlocked Mac in a shared workspace.
- A family member or friend borrows the laptop and opens personal applications such as Messages, Photos, or Notes.
- Sensitive applications (banking, email, code editors with proprietary projects) remain unprotected after login.
- Existing third-party solutions often require cloud connectivity, raising privacy and data sovereignty concerns.

FaceLock Pro addresses these gaps by providing application-level biometric protection that operates entirely on-device.

1.3 Project Objectives

The core objectives include:

1. **Application Protection:** Allow users to select and protect any installed application.

2. **Facial Recognition:** Unlock applications using real-time AI-powered facial recognition.
3. **Access Prevention:** Prevent unauthorized access to protected applications by intercepting launch events.
4. **Offline Operation:** Ensure 100% offline processing to guarantee user privacy.
5. **Secure Storage:** Store biometric data (face embeddings) encrypted locally using the macOS Keychain.
6. **Fast Authentication:** Provide sub-500-millisecond authentication using Apple Neural Engine hardware acceleration.
7. **Battery Efficiency:** Minimize battery and CPU impact through event-driven architecture.
8. **Native Experience:** Adhere to Apple’s macOS Human Interface Guidelines for a polished user experience.

1.4 Scope

1.4.1 In Scope

- Native macOS application for Apple Silicon.
- Offline facial recognition using ArcFace (InsightFace) converted to Core ML.
- Multi-factor authentication (Face, Touch ID, macOS password, PIN).
- Background application monitoring and protection.
- Face enrollment wizard with quality validation.
- Liveness detection (anti-spoofing).
- Activity logging and native notifications.
- Configurable settings for security, sessions, and privacy.

1.4.2 Out of Scope

- Cloud-based authentication or biometric sync.
- Mac App Store distribution (requires sandboxing, which prevents app monitoring).
- Intel Mac support (Apple Neural Engine required).
- System login screen replacement.
- File or folder-level encryption (planned for future versions).

Chapter 2

Core Features

2.1 AI Face Unlock

The primary authentication method uses real-time facial recognition processed entirely on-device.

2.1.1 Technical Details

- **Camera Input:** Live feed captured via `AVFoundation` using `AVCaptureSession` with a front-facing camera (`.builtInWideAngleCamera`, position `.front`).
- **Face Detection:** Apple’s Vision Framework (`VNDetectFaceRectanglesRequest`) provides hardware-accelerated face bounding box detection in under 100 ms.
- **Landmark Detection:** `VNDetectFaceLandmarksRequest` extracts 76 facial landmarks (eyes, nose, mouth, jawline, eyebrows) for alignment.
- **Face Alignment:** An affine transformation is computed from eye positions to normalize the face to a frontal 112×112 pixel image.
- **Feature Extraction:** The aligned face is passed through the ArcFace model (converted to Core ML), which outputs a 512-dimensional embedding vector.
- **Comparison:** Cosine similarity is computed between the live embedding and the stored reference embedding. If the similarity exceeds the configurable threshold (default: 0.55), authentication succeeds.

2.1.2 Cosine Similarity

The similarity between two embedding vectors **a** and **b** is calculated as:

$$\text{similarity}(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \times \|\mathbf{b}\|} = \frac{\sum_{i=1}^{512} a_i \cdot b_i}{\sqrt{\sum_{i=1}^{512} a_i^2} \times \sqrt{\sum_{i=1}^{512} b_i^2}}$$

This computation is accelerated using Apple’s `Accelerate` framework (`vDSP`) for SIMD vector operations on Apple Silicon.

2.2 Application Locker

Users can protect any installed macOS application by selecting it from a list.

2.2.1 App Discovery

Installed applications are discovered by scanning the `/Applications` directory and user-specific `~/Applications`. For each application bundle (`.app`), the following metadata is extracted:

- Display name (from `CFBundleName` in `Info.plist`)
- Bundle identifier (e.g., `com.apple.Safari`)
- Application icon (via `NSWorkspace.shared.icon(forFile:)`)
- File path

2.2.2 Data Model

Each protected application is persisted with the following schema:

Field	Type	Description
id	UUID	Unique identifier
name	String	Display name
bundleIdentifier	String	macOS bundle ID
path	String	Path to <code>.app</code> bundle
isProtected	Bool	Whether currently protected
dateAdded	Date	When app was added to list

Table 2.1: ProtectedApp data model

2.3 Background Protection

The application continuously monitors running processes using macOS system APIs.

2.3.1 Monitoring Mechanism

Two complementary methods are used for reliability:

1. **NSWorkspace Notifications:** The app subscribes to `NSWorkspace.didLaunchApplicationNot` via `NSWorkspace.shared.notificationCenter`. When any application launches, the notification provides an `NSRunningApplication` object containing the bundle identifier.
2. **Key-Value Observing (KVO):** As a backup, the app observes `NSWorkspace.shared.runningAp` using KVO (`observe(\.runningApplications)`) to detect applications that may not trigger the standard notification (e.g., background-only apps).

2.3.2 Interception Flow

When a protected application is detected:

1. The bundle identifier is checked against the protected apps list.
2. If the user has an active authenticated session for that app, access is granted immediately.
3. Otherwise, the protected application is hidden using `NSRunningApplication.hide()`.
4. A floating authentication panel (`NSPanel` with `.floating` window level) is presented.
5. Upon successful authentication, the app is unhidden via `NSRunningApplication.unhide()`.
6. On repeated failures (exceeding max attempts), the app may be terminated via `NSRunningApplication.terminate()`.

2.3.3 macOS API Constraints

- The application must be **non-sandboxed** to call `hide()`, `unhide()`, and `terminate()` on other applications. Sandboxed apps are prohibited from controlling other processes.
- This means the application **cannot be distributed via the Mac App Store**.
- Distribution is via notarized DMG, direct download, or Homebrew cask.
- A determined user could bypass hiding via Mission Control; this is documented as a known limitation.

2.4 Face Enrollment Wizard

The enrollment wizard guides the user through capturing their facial data for future authentication.

2.4.1 Process

1. **Pre-checks:** Camera permission is verified. Ambient lighting is assessed using the camera's average luminance. The user is prompted to ensure good lighting and remove obstructions (sunglasses, masks).
2. **Capture Phase:** 20–30 face images are captured at different angles:
 - Center (frontal)
 - Slight left, slight right
 - Slight up, slight down
 - Tilted left, tilted right

Each captured frame undergoes quality validation:

- Face detected within bounding box ($> 70\%$ of frame)
- Face capture quality score > 0.5 (via `VNDetectFaceCaptureQualityRequest`)
- Both eyes visible (via landmark detection)
- Sufficient image sharpness (Laplacian variance check)

3. **Embedding Generation:** Each valid capture is aligned and passed through the ArcFace model to produce a 512-dimensional embedding vector.
4. **Averaging:** All embeddings are element-wise averaged to produce a single reference embedding:

$$\mathbf{e}_{\text{ref}} = \frac{1}{N} \sum_{i=1}^N \mathbf{e}_i$$

where N is the number of valid captures and $\mathbf{e}_i$ is the embedding for capture i .

5. **Normalization:** The averaged embedding is L2-normalized:

$$\hat{\mathbf{e}}_{\text{ref}} = \frac{\mathbf{e}_{\text{ref}}}{\|\mathbf{e}_{\text{ref}}\|}$$

6. **Encryption & Storage:** The normalized embedding is serialized, encrypted, and stored in the macOS Keychain under a dedicated key.
7. **Cleanup:** All raw image data and intermediate embeddings are zeroed out and deallocated from memory. No images are ever written to disk.

2.5 Liveness Detection

Liveness detection prevents spoofing attacks where an adversary presents a photo or video of the enrolled user.

2.5.1 Tier 1: Vision Framework (No Additional Model)

The primary liveness detection uses Apple’s Vision Framework landmarks:

- **Blink Detection:** The Eye Aspect Ratio (EAR) is computed from eye landmarks. EAR drops below a threshold (~ 0.2) during a blink. Two consecutive blinks within 3 seconds confirms liveness.

$$\text{EAR} = \frac{|p_2 - p_6| + |p_3 - p_5|}{2 \times |p_1 - p_4|}$$

where $p_1 \dots p_6$ are the six landmarks defining each eye contour.

- **Head Pose Estimation:** Yaw, pitch, and roll angles are computed from facial landmarks. The system issues random challenges:
 - “Look left” (yaw < -15)
 - “Look right” (yaw > 15)
 - “Look up” (pitch > 10)
 - “Nod down” (pitch < -10)
- **Multi-Frame Consistency:** The system verifies that face landmarks change naturally across multiple consecutive frames rather than remaining static (as with a photograph).

2.5.2 Tier 2: MiniFASNet (Optional Advanced Model)

An optional anti-spoofing classification model:

- **Model:** MiniFASNet from the Silent-Face-Anti-Spoofing project.
- **Input:** 80×80 pixel face crop (BGR, normalized to [0, 1]).
- **Output:** Binary classification — real face vs. spoof (photo/screen replay).
- **Conversion:** PyTorch → TorchScript → Core ML via `coremltools`.
- **Usage:** Runs in parallel with ArcFace; both must pass for authentication to succeed.

2.6 Multi-Factor Authentication System

FaceLock Pro implements a cascading authentication system with four methods.

2.6.1 Authentication Methods

Method	Description	Framework	Data Storage
Face Unlock	Custom ArcFace recognition	Vision + Core ML	Keychain (encrypted)
Touch ID	Native fingerprint sensor	LocalAuthentication	Secure Enclave
macOS Password	System login password	LocalAuthentication	Managed by macOS
Application PIN	4–8 digit user PIN	Custom	Keychain (hashed)

Table 2.2: Supported authentication methods

2.6.2 Authentication Cascade

When a protected application is launched, the following cascade is executed:

1. Face Unlock is attempted automatically (camera activates).
2. If Face Unlock succeeds → access is granted immediately.
3. If Face Unlock fails or times out (5 seconds) → Touch ID is offered.
4. If Touch ID is unavailable or cancelled → macOS password dialog is presented.
5. If the user has enabled a PIN → PIN entry is offered as a final fallback.
6. If all methods fail or are cancelled → access is denied; the app remains locked.

2.6.3 Touch ID and macOS Password Integration

Both Touch ID and macOS password authentication are implemented using Apple's LocalAuthentication framework:

Listing 2.1: Touch ID / macOS Password via LAContext

```
import LocalAuthentication

let context = LAContext()
var error: NSError?

// Check biometric availability
if context.canEvaluatePolicy(
    .deviceOwnerAuthenticationWithBiometrics, error: &error) {

    context.evaluatePolicy(
        .deviceOwnerAuthenticationWithBiometrics,
        localizedReason: "Unlock protected application"
    ) { success, error in
        // Handle result
    }
}
```

The application never accesses, stores, or transmits fingerprint data. All biometric verification is handled by the Secure Enclave through macOS system APIs.

2.6.4 PIN Authentication

- PIN length: 4–8 digits, configurable by the user.
- Storage: The PIN is salted, hashed (SHA-256), and stored in the macOS Keychain.
- Retry limits: Configurable maximum attempts (default: 5).
- Cooldown: After max failures, a 60-second cooldown period is enforced.
- The raw PIN is never stored; only the salted hash is persisted.

2.7 Session Management

To avoid requiring authentication on every interaction, FaceLock Pro implements session management.

2.7.1 Session Timeout Options

Option	Duration
Always Authenticate	0 seconds (every access)
5 Minutes	300 seconds
10 Minutes	600 seconds
30 Minutes	1,800 seconds
1 Hour	3,600 seconds
Until Logout	Indefinite (until macOS session ends)

Table 2.3: Configurable session timeout durations

2.7.2 Auto-Lock Triggers

Sessions are invalidated and re-authentication is required when:

- The Mac enters sleep (`NSWorkspace.willSleepNotification`).
- The screen saver activates (`NSWorkspace.screensDidSleepNotification`).
- The session timeout expires.
- The user manually locks via the menu bar (future feature).

2.8 Activity Logging

Every authentication event is recorded for audit purposes.

2.8.1 Event Types

Event	Icon	Description
App Opened	arrow.up.forward.app	A protected app was launched
Auth Success	checkmark.shield.fill	Authentication succeeded
Auth Failure	xmark.shield.fill	Authentication failed
App Protected	lock.fill	An app was added to protected list
App Unprotected	lock.open.fill	An app was removed from list
Enrollment Completed	person.crop.circle.badge.checkmark	Face data enrolled
Settings Changed	gearshape.fill	Security settings modified
Session Expired	clock.badge.xmark	Trusted session timed out
Lockout Triggered	exclamationmark.lock.fill	Max failed attempts exceeded

Table 2.4: Activity log event types

2.8.2 Log Entry Schema

Each log entry stores:

Field	Type	Description
id	UUID	Unique identifier
timestamp	Date	Event timestamp
eventType	EventType	Category of event
appBundleIdentifier	String?	Target app’s bundle ID
appName	String?	Target app’s display name
authMethod	AuthMethod?	Method used (Face/TouchID/Password/PIN)
wasSuccessful	Bool	Whether the action succeeded
confidenceScore	Float?	Face recognition confidence (0.0–1.0)
details	String?	Additional context or error message

Table 2.5: ActivityLogEntry data model

2.9 Notifications

The application uses macOS native notifications via the `UserNotifications` framework (`UNUserNotificationCenter`):

- Authentication succeeded for a protected app.
- Authentication failed (with attempt count).
- New application added to the protected list.
- Lockout triggered after maximum failed attempts.
- Security settings were modified.

2.10 Settings and Configuration

The application provides comprehensive user-configurable settings organized into four categories:

Category	Settings	Storage
General	Launch at startup, start minimized, theme (system/light/dark)	UserDefaults
Security	Auth methods toggle, confidence threshold (0.3–0.9), max failed attempts (3–10), lock after sleep, session timeout	UserDefaults
Camera	Camera device selection, liveness detection toggle	UserDefaults
Privacy	Clear activity logs, delete face data, export/import settings	UserDefaults + Keychain

Table 2.6: Settings categories and storage

Chapter 3

Technical Architecture

3.1 Design Pattern: MVVM

The application follows the Model-View-ViewModel (MVVM) architecture pattern:

- **Model:** Plain Swift structs/classes representing data (e.g., `ProtectedApp`, `FaceEmbedding`, `ActivityLogEntry`). Models are `Codable` for persistence and `Identifiable` for SwiftUI list rendering.
- **View:** SwiftUI views responsible only for rendering UI and forwarding user actions to ViewModels. Views use `@EnvironmentObject`, `@StateObject`, and `@Binding` for state management.
- **ViewModel:** `ObservableObject` classes that encapsulate business logic, transform data for display, and coordinate between services/managers. ViewModels publish state changes via `@Published` properties.

3.2 Layered Architecture

The application is divided into five distinct layers, each with clear responsibilities:

3.2.1 Layer 1: Presentation Layer (SwiftUI + AppKit)

- All user interface views built with SwiftUI.
- `NavigationSplitView` for main sidebar navigation.
- `NSPanel` (AppKit) for the floating unlock dialog.
- `NSApplicationDelegateAdaptor` bridges SwiftUI and AppKit lifecycles.
- Screens: Welcome, Dashboard, Protected Apps, Enrollment, History, Settings, Unlock, About.

3.2.2 Layer 2: Business Logic Layer (Managers)

Manager	Responsibility
AuthenticationManager	Orchestrates the auth cascade (Face → TouchID → Password → PIN)
FaceRecognitionManager	Manages ArcFace inference, embedding comparison, threshold logic
LivenessManager	Generates challenges, tracks completion, validates liveness
AppMonitorService	Subscribes to NSWorkspace notifications, intercepts protected app launches
SessionManager	Tracks per-app sessions, handles timeout and auto-lock
NotificationManager	Sends native macOS notifications for auth events
SettingsManager	Reads/writes user preferences to UserDefaults
ActivityLogManager	Records events to SQLite/SwiftData database

Table 3.1: Business logic managers

3.2.3 Layer 3: Machine Learning Layer

Component	Responsibility	Framework
FaceDetector	Bounding box + landmark detection	Vision Framework
FaceAligner	Affine alignment to 112×112	Core Graphics
EmbeddingGenerator	ArcFace inference → 512-dim vector	Core ML
CosineSimilarity	Vector comparison using SIMD	Accelerate (vDSP)
LivenessClassifier	Real vs. spoof classification (optional)	Core ML (MiniFASNet)

Table 3.2: Machine learning components

3.2.4 Layer 4: Storage Layer

Store	Data	Technology
Keychain	Face embeddings (encrypted), PIN (hashed), secrets	Security Framework
Database	Activity logs, protected apps list	SQLite / SwiftData
UserDefaults	User preferences, settings	Foundation

Table 3.3: Storage layer components

3.2.5 Layer 5: System Layer

Framework	Usage
NSWorkspace	App launch monitoring, sleep/wake notifications, running apps list
AVFoundation	Camera capture session, live video preview, frame extraction
LocalAuthentication	Touch ID and macOS password verification via LAContext
UserNotifications	Native notification delivery (UNUserNotificationCenter)

Table 3.4: System layer frameworks

3.3 Project File Structure

The Xcode project is generated from a `project.yml` specification using XcodeGen:

Listing 3.1: Project directory structure

```

AppLockPro/
  App/
    AppLockProApp.swift           // @main SwiftUI entry point
    AppDelegate.swift             // NSApplicationDelegate
    AppState.swift                // Shared observable state
    Info.plist                    // Camera usage, app metadata
    AppLockPro.entitlements        // Camera entitlement
  Models/
    ProtectedApp.swift            // Protected + installed app
    FaceEmbedding.swift           // 512-dim embedding container
    AuthMethod.swift              // Auth method enum
    AuthenticationResult.swift     // Auth attempt result
    ActivityLogEntry.swift         // Log entry + event types
    AppSettings.swift             // Settings + session timeout
  Views/
    ContentView.swift             // Root router
    Components/SidebarView.swift
    Welcome/WelcomeView.swift
    Dashboard/DashboardView.swift
    ProtectedApps/ProtectedAppsView.swift
    Enrollment/EnrollmentWizardView.swift
    History/ActivityHistoryView.swift
    Settings/SettingsView.swift
    Unlock/UnlockDialogView.swift
    About/AboutView.swift
  ViewModels/                    // (Milestone 2+)
  Managers/                      // (Milestone 4+)
  Services/                      // (Milestone 2+)
  AI/                            // (Milestone 3+)
  Utilities/
    Constants.swift               // App-wide constants
    Logger.swift                  // os.Logger categories

```

```
Resources/
```

```
Assets.xcassets
```

```
// AccentColor, AppIcon
```

Chapter 4

AI Pipeline

4.1 Pipeline Overview

The complete AI authentication pipeline executes the following stages sequentially:

1. **Camera Capture** — AVFoundation provides a `CMSampleBuffer` from the front camera at 30 FPS.
2. **Face Detection** — Vision Framework's `VNDetectFaceRectanglesRequest` locates faces in the frame.
3. **Landmark Extraction** — `VNDetectFaceLandmarksRequest` identifies 76 facial landmarks.
4. **Quality Assessment** — `VNDetectFaceCaptureQualityRequest` scores the face quality (0.0–1.0).
5. **Face Alignment** — An affine transform normalizes the face using eye positions to a 112×112 image.
6. **Liveness Check** — Eye aspect ratio and head pose are analyzed across multiple frames.
7. **Feature Extraction** — The aligned face is fed to ArcFace (Core ML), producing a 512-dim vector.
8. **L2 Normalization** — The embedding is normalized to unit length.
9. **Cosine Similarity** — The normalized embedding is compared against the stored reference.
10. **Decision** — If similarity $\geq$ threshold (default 0.55), authentication succeeds.

4.2 ArcFace Model

4.2.1 Model Specifications

Property	Value
Architecture	ResNet-100 (ArcFace loss)
Source	InsightFace (open source)
Input Size	$112 \times 112 \times 3$ (RGB)
Output Size	512-dimensional float vector
Original Format	PyTorch (.pth) / ONNX
Target Format	Core ML (.mlpackage)
Inference Hardware	Apple Neural Engine (ANE)
Expected Latency	< 200 ms on Apple Silicon

Table 4.1: ArcFace model specifications

4.2.2 Model Conversion Pipeline

The conversion from PyTorch to Core ML follows this workflow:

1. Download pre-trained ArcFace-R100 weights from the InsightFace model zoo.
2. Load the PyTorch model and set to evaluation mode.
3. Create a dummy input tensor of shape (1, 3, 112, 112).
4. Trace the model using `torch.jit.trace()`.
5. Convert to Core ML using `coremltools.convert()` with image input type.
6. Verify output shape is (1, 512).
7. Save as `.mlpackage` and add to the Xcode project.

Listing 4.1: ArcFace model conversion script

```
import torch
import coremltools as ct

# Load pretrained model
model = load_arcface_r100("arcface_r100.pth")
model.eval()

# Trace with dummy input
dummy = torch.randn(1, 3, 112, 112)
traced = torch.jit.trace(model, dummy)

# Convert to Core ML
mlmodel = ct.convert(
    traced,
    inputs=[ct.ImageType(
        name="faceImage",
        shape=(1, 3, 112, 112),
        scale=1.0/127.5,
```

```

        bias=[-1, -1, -1]
    )],
    minimum_deployment_target=ct.target.macOS14
)
mlmodel.save("ArcFace.mlpackage")

```

4.2.3 Preprocessing Requirements

Before passing a face image to the ArcFace model:

1. Detect face bounding box using Vision Framework.
2. Extract eye landmark positions.
3. Compute affine transformation to align eyes horizontally.
4. Apply the transform and crop to 112×112 pixels.
5. Convert color space to RGB.
6. Normalize pixel values to $[-1, 1]$ (scale by $1/127.5$, bias -1).
7. Convert to `CVPixelBuffer` for Core ML input.

4.3 MiniFASNet Anti-Spoofing Model

Property	Value
Architecture	MiniFASNet (compact CNN)
Source	Silent-Face-Anti-Spoofing (GitHub)
Input Size	$80 \times 80 \times 3$ (BGR)
Output	2-class (real / spoof)
Preprocessing	Face crop with $2.7 \times$ margin, normalize by 255
Original Format	PyTorch (.pth)
Target Format	Core ML (.mlpackage)

Table 4.2: MiniFASNet model specifications

4.4 Hardware Acceleration

All Core ML models are automatically dispatched to the most efficient compute unit by the Core ML runtime:

- **Apple Neural Engine (ANE):** Primary target for both ArcFace and MiniFASNet. Provides the fastest inference with lowest power consumption.
- **GPU (Metal):** Fallback when ANE is unavailable or when the model contains unsupported operations.
- **CPU:** Final fallback; slowest but always available.

The `MLModelConfiguration` is set to `.all` compute units, allowing Core ML to choose the optimal hardware at runtime.

Chapter 5

Technology Stack

5.1 Complete Technology Table

Component	Technology	Purpose
Programming Language	Swift 5.9+	Type-safe, performant, native
UI Framework	SwiftUI	Declarative, modern interface
App Framework	AppKit	Window management, system integration
Camera	AVFoundation	Real-time camera capture
Face Detection	Vision Framework	Hardware-accelerated detection
Face Recognition	ArcFace (Core ML)	512-dim embedding extraction
Liveness Detection	Vision + MiniFASNet	Anti-spoofing
Hardware Acceleration	Apple Neural Engine	Fast on-device ML inference
Vector Math	Accelerate (vDSP)	SIMD cosine similarity
Auth Fallback	LocalAuthentication	Touch ID + macOS password
Secrets	Keychain Services	Encrypted embedding storage
Database	SQLite / SwiftData	Activity logs, app list
App Monitoring	NSWorkspace	App launch detection
Notifications	UserNotifications	Native macOS alerts
Logging	os.Logger	Unified system logging
Project Generation	XcodeGen	.xcodeproj from YAML
Build System	xcodebuild	Command-line builds
Version Control	Git	Source code management

Table 5.1: Complete technology stack

5.2 System Requirements

Requirement	Specification
Operating System	macOS 14.0 (Sonoma) or later
Processor	Apple Silicon (M1, M2, M3, M4)
Camera	Built-in FaceTime HD or compatible external
Build Tool	Xcode 15.0+
Project Generator	XcodeGen 2.x
Disk Space	~50 MB (app) + ~100 MB (Core ML models)
RAM	< 200 MB runtime footprint

Table 5.2: System requirements

5.3 Build and Run Instructions

Listing 5.1: Building from the command line

```
# Install XcodeGen (one-time)
brew install xcodegen

# Clone the repository
git clone https://github.com/ThanojBuddhima/applockPro.git
cd applockPro

# Generate the Xcode project from project.yml
xcodegen generate

# Build the application
xcodebuild build \
  -project AppLockPro.xcodeproj \
  -scheme AppLockPro \
  -configuration Debug \
  -destination 'platform=macOS'

# Run the built app
open ~/Library/Developer/Xcode/DerivedData/AppLockPro-*/\
  Build/Products/Debug/FaceLock\ Pro.app
```

Alternatively, open AppLockPro.xcodeproj in Xcode and press Cmd+R.

Chapter 6

Security Design

6.1 Security Principles

1. **Zero Cloud Upload:** All biometric data, images, and embeddings remain on the local device. No network requests are made for authentication.
2. **No Image Persistence:** Raw face images captured during enrollment or authentication are never written to disk. They exist only in memory and are deallocated immediately after embedding generation.
3. **Keychain-Only Secrets:** Face embeddings and PINs are stored exclusively in the macOS Keychain, which provides hardware-backed AES-256 encryption via the Secure Enclave.
4. **Local Inference:** All AI models run on-device via Core ML, utilizing the Apple Neural Engine.
5. **Hardened Runtime:** The application is built with Hardened Runtime enabled, which restricts code injection, memory access, and dynamic library loading.
6. **Principle of Least Privilege:** The application only requests the permissions it needs (camera access, accessibility for app monitoring).

6.2 Data Protection Matrix

Data Type	Storage	Encryption	Lifecycle
Face embeddings	Keychain	AES-256 (hardware)	Persisted until user deletes
Application PIN	Keychain	SHA-256 hash + salt	Persisted until user changes
Activity logs	SQLite	Application-level	Clearable by user
Protected apps list	SQLite	Application-level	User-managed
User preferences	UserDefaults	None (non-sensitive)	Persisted
Raw face images	RAM only	N/A	Deleted immediately
Camera frames	RAM only	N/A	Not retained

Table 6.1: Data protection matrix

6.3 Threat Model

Threat	Attack Vector	Mitigation
Photo replay	Attacker holds a photo of the user in front of the camera	Liveness detection: blink detection, head movement challenges, multi-frame consistency
Video replay	Attacker plays a video of the user on a screen	MiniFASNet anti-spoofing classifier, random challenge sequence
Embedding theft	Attacker extracts stored face embeddings from disk	Keychain AES-256 encryption, Hardened Runtime prevents memory inspection
Brute-force PIN	Attacker repeatedly guesses the PIN	Max attempts (default 5), cooldown period (60s), temporary lockout
App bypass	User unhides protected app via Mission Control	Documented limitation; future mitigation via Endpoint Security Framework
Process injection	Attacker injects code to bypass authentication checks	Hardened Runtime, code signing, non-sandboxed but signed with Developer ID

Table 6.2: Threat model and mitigations

6.4 Keychain Integration

The macOS Keychain is used for all sensitive data storage:

Listing 6.1: Keychain storage example

```
import Security

func storeEmbedding(_ data: Data, forKey key: String) -> Bool {
    let query: [String: Any] = [
        kSecClass as String: kSecClassGenericPassword,
        kSecAttrAccount as String: key,
        kSecValueData as String: data,
        kSecAttrAccessible as String:
            kSecAttrAccessibleWhenUnlockedThisDeviceOnly
    ]

    // Delete existing item if present
    SecItemDelete(query as CFDictionary)

    // Add new item
    let status = SecItemAdd(query as CFDictionary, nil)
    return status == errSecSuccess
}
```

The `kSecAttrAccessibleWhenUnlockedThisDeviceOnly` flag ensures embeddings are only accessible when the device is unlocked and cannot be transferred to another device via backup.

Chapter 7

User Interface Design

7.1 Design Principles

The interface follows Apple’s macOS Human Interface Guidelines:

- Native macOS look and feel using SwiftUI.
- Support for Light Mode and Dark Mode.
- Smooth animations (spring, easeInOut).
- SF Symbols for consistent iconography.
- Keyboard shortcut support.
- Accessibility labels for VoiceOver compatibility.
- Retina display optimization.

7.2 Application Screens

7.2.1 Welcome / Onboarding (4 Steps)

A guided wizard presented on first launch:

1. **Welcome:** Introduction with feature highlights (app protection, face unlock, offline privacy).
2. **Permissions:** Camera and accessibility permission requests with status indicators.
3. **Enrollment Intro:** Explanation of the face capture process.
4. **Complete:** Confirmation that setup is finished; “Get Started” button.

Progress is shown via animated capsule indicators at the top.

7.2.2 Dashboard

The main screen after onboarding, featuring:

- **Status Badge:** Shows enrollment status (“Face Enrolled” / “Not Enrolled”).
- **Stat Cards:** Protected apps count, successful unlocks today, failed attempts today, session status.
- **Quick Actions:** Add app, re-enroll face, view history, settings.
- **Recent Activity:** Last 5 authentication events.

7.2.3 Protected Apps

- Search bar to filter protected apps.
- List of protected apps with name, bundle ID, icon, and toggle switch.
- “Add App” button opens a sheet with all installed applications.
- Swipe-to-delete for removing apps from protection.

7.2.4 Face Enrollment Wizard

Five-state wizard:

1. **Ready:** Instructions and lighting requirements.
2. **Capturing:** Live camera preview with angle guidance and progress bar.
3. **Processing:** Spinner while embeddings are generated and encrypted.
4. **Complete:** Success confirmation.
5. **Error:** Failure message with “Try Again” option.

7.2.5 Unlock Dialog

A floating panel (`NSPanel`) displayed above all windows:

- Shows the requesting application’s name and icon.
- Camera preview for Face Unlock (primary method).
- Method switcher: buttons to switch between Face, Touch ID, Password, and PIN.
- Authentication status feedback (authenticating, success, failure, locked out).

7.2.6 Activity History

- Filter chips for each event type (All, Auth Success, Auth Failure, etc.).
- Chronological list with event icon, name, app name, confidence score, and relative timestamp.
- “Clear All” button with confirmation dialog.

7.2.7 Settings

Four-tab interface:

- **General:** Launch at startup, start minimized, theme (System/Light/Dark).
- **Security:** Auth method toggles, confidence threshold slider (30%–90%), max attempts stepper, lock after sleep, session timeout picker.
- **Camera:** Camera device picker, liveness detection toggle.
- **Privacy:** Clear logs, delete face data (with confirmation dialogs), export/import settings.

7.2.8 About

Application information: icon, name, version, description, technology badges, and copyright.

7.3 Navigation Architecture

The application uses `NavigationSplitView` with a sidebar containing three sections:

- **Main:** Dashboard, Protected Apps
- **Security:** Face Enrollment, Activity History
- **Application:** Settings, About

Navigation selection is tracked via the shared `AppState` object, which is injected into the view hierarchy using `@EnvironmentObject`.

Chapter 8

Performance

8.1 Performance Targets

Metric	Target	Method	Hardware
Face detection	< 100 ms	Vision Framework	GPU
Face recognition	< 200 ms	ArcFace Core ML	Apple Neural Engine
Cosine similarity	< 1 ms	Accelerate vDSP	CPU (SIMD)
Total unlock time	< 500 ms	End-to-end pipeline	ANE + GPU + CPU
CPU usage (idle)	< 10%	Event-driven monitoring	CPU
Memory footprint	< 200 MB	Model + UI + buffers	RAM
Battery impact	Minimal	ANE offloading	Neural Engine
App monitoring	< 1 ms	NSWorkspace notifications	CPU (event)

Table 8.1: Performance targets with hardware mapping

8.2 Optimization Strategies

- **Event-Driven Monitoring:** App monitoring uses NSWorkspace notifications rather than polling, consuming near-zero CPU when no protected app is launched.
- **Lazy Model Loading:** Core ML models are loaded into memory only when face authentication is first requested, not at app startup.
- **Frame Skipping:** During live camera preview, face detection is performed on every 3rd frame (10 FPS) rather than every frame (30 FPS) to reduce CPU/GPU load.
- **Accelerate Framework:** Cosine similarity computation uses `vDSP.dot()` and `vDSP.sumOfSquares()` for SIMD-optimized vector operations.
- **Memory Management:** Camera frame buffers are released immediately after processing. Auto-release pools are used for tight loops.

Chapter 9

Implementation Roadmap

Development is divided into seven milestones, each producing a functional increment:

9.1 Milestone 1: Project Setup & UI Shell (Complete)

- Xcode project generation via XcodeGen.
- Complete folder structure (App, Models, Views, ViewModels, Managers, Services, AI, Utilities).
- All SwiftUI screens implemented as navigable stubs.
- Data models for all entities.
- Build verification: `xcodebuild` succeeds.

9.2 Milestone 2: Camera & Face Detection

- `CameraService`: AVFoundation capture session with front camera.
- Camera preview via `AVCaptureVideoPreviewLayer` wrapped in `NSViewRepresentable`.
- `FaceDetector`: Vision Framework face detection and landmark extraction.
- Real-time face bounding box overlay on camera feed.
- `FaceAligner`: Affine alignment to 112×112 using eye positions.
- Face quality validation (lighting, sharpness, eyes visible).

9.3 Milestone 3: AI Face Recognition

- Python script for ArcFace PyTorch → Core ML conversion.
- `EmbeddingGenerator`: Core ML inference wrapper.
- `CosineSimilarity`: Accelerate-powered vector comparison.
- Enrollment pipeline: capture → detect → align → embed → average → encrypt → store.
- Authentication pipeline: capture → detect → align → embed → compare → decision.

9.4 Milestone 4: App Monitoring & Protection

- `AppMonitorService`: `NSWorkspace` notifications + KVO.
- Installed apps browser (scan `/Applications`).
- App hide/terminate on unauthorized launch.
- Floating unlock panel (`NSPanel`).
- Session management with configurable timeouts.
- Sleep/screen lock event handling.

9.5 Milestone 5: Multi-Factor Authentication

- Touch ID + macOS password via `LAContext`.
- PIN setup, verification, and Keychain storage.
- Authentication cascade logic.
- `KeychainService` wrapper.
- Activity logging to `SQLite/SwiftData`.
- Failed attempt tracking, cooldown, and lockout.
- Native macOS notifications.

9.6 Milestone 6: Liveness Detection

- Blink detection via eye aspect ratio.
- Head pose estimation from landmarks.
- Random challenge generator and UI.
- Multi-frame consistency verification.
- Optional MiniFASNet integration.

9.7 Milestone 7: Polish & Release

- Smooth animations and transitions.
- App icon design.
- Keyboard shortcuts and accessibility.
- Performance profiling with Instruments.
- Error handling for all edge cases.
- Code signing with Developer ID.
- Notarization for Gatekeeper.
- DMG packaging.
- Documentation (README, architecture doc, user guide).

9.8 Estimated Timeline

Week	Milestone	Deliverable
1	Setup & Learning	Project structure, SwiftUI basics
2	Milestone 1 (complete)	Full UI skeleton, all screens navigable
3	Milestone 2	Live camera + face detection
4	Milestone 3	ArcFace integration, enrollment, auth
5	Milestone 4	App monitoring, protection, unlock flow
6	Milestone 5	Touch ID, PIN, logging, notifications
7	Milestone 6	Liveness detection
8	Milestone 7	Polish, testing, release candidate

Table 9.1: Development timeline

Chapter 10

Testing Strategy

10.1 Unit Tests

- **CosineSimilarity**: Verify correct similarity scores for known vectors, edge cases (zero vectors, identical vectors, orthogonal vectors).
- **EmbeddingGenerator**: Test with mock Core ML model, verify output dimensions.
- **KeychainService**: Store/retrieve/delete operations with test keys.
- **SessionManager**: Timeout expiration, auto-lock triggers, session creation/invalidation.
- **ActivityLogManager**: Event recording, filtering, clearing.
- **AppMonitorService**: Protected app lookup, notification handling.

10.2 Integration Tests

- Full enrollment pipeline: 20 captures → embeddings → average → Keychain storage.
- Full authentication pipeline: camera → detection → alignment → embedding → comparison.
- Authentication cascade: Face failure → Touch ID → Password → PIN.
- App monitoring: Launch protected app → detect → block → authenticate → unlock.

10.3 Manual Verification Checklist

- Camera preview displays live feed.
- Face detection draws bounding box in real-time.
- Enrollment captures faces and stores encrypted embedding.
- Face unlock succeeds for enrolled user within 500 ms.
- Face unlock rejects unknown faces.
- Protected app is hidden on unauthorized launch.
- Protected app unlocks after successful authentication.
- Touch ID and PIN fallbacks work correctly.
- Auto-lock triggers on sleep.
- Activity log records all events.
- Settings persist across app restarts.

- Launch at login works.
- CPU stays below 10% at idle.
- Memory stays under 200 MB.

Chapter 11

Risk Assessment

Risk	Likelihood	Impact	Mitigation
ArcFace model conversion fails	Medium	High	Fallback: use MobileFaceNet or ONNX Runtime with CoreML Execution Provider
App hiding is bypassable	High	Medium	Document limitation; plan Endpoint Security Framework upgrade in v2
Camera permission denied	Low	Medium	Graceful fallback to Touch ID / password only
Performance exceeds targets	Medium	Medium	Profile early with Instruments; use ANE; reduce model size if needed
MiniFASNet conversion issues	Medium	Low	Defer to Vision-only liveness (Tier 1) which needs no extra model
Keychain access issues	Low	Low	Proper entitlements; test on clean macOS install
ArcFace licensing conflict	Low	High	Use non-commercial for learning; purchase license or switch model for commercial release

Table 11.1: Risk assessment matrix

Chapter 12

Future Enhancements

Future versions of FaceLock Pro may include:

- **Multiple User Profiles:** Allow multiple users to enroll and authenticate on a shared Mac.
- **Apple Watch Proximity Unlock:** Use Bluetooth proximity with a paired Apple Watch to unlock apps without camera.
- **Geofencing:** Automatically disable authentication at trusted locations (e.g., home, office) using Core Location.
- **Secure File Vault:** Encrypt and protect individual files and folders with face authentication.
- **Endpoint Security Framework:** Intercept process launches at the kernel level for stronger app blocking (requires Apple entitlement).
- **Encrypted Backups:** Export and import encrypted face data and settings for device migration.
- **AI Anomaly Detection:** Detect unusual authentication patterns (e.g., repeated failures from an unknown face).
- **Plugin Architecture:** Allow third-party extensions for custom protection rules.
- **Enterprise Management:** MDM integration for corporate deployment.
- **Menu Bar Controls:** Quick lock/unlock, status, and settings from the macOS menu bar.
- **Browser Extension:** Protect browser tabs and bookmarks within Safari and Chrome.
- **Clipboard Protection:** Clear clipboard contents after a configurable timeout.
- **Remote Lock:** Lock all protected apps remotely via a companion iOS app.

Chapter 13

Conclusion

FaceLock Pro addresses a real gap in macOS security by providing application-level biometric protection that is private, fast, and entirely offline. By leveraging Apple Silicon’s Neural Engine, the Vision framework, and the ArcFace facial recognition model, the application achieves sub-500-millisecond authentication without compromising user privacy.

The modular MVVM architecture, phased development approach, and clear separation of concerns ensure that the project remains maintainable as new features are added. The milestone-based roadmap transforms a complex undertaking into a series of achievable increments, each delivering a working product.

This project demonstrates proficiency in Swift, SwiftUI, Core ML, computer vision, biometric security, and native macOS development — making it a comprehensive showcase of modern Apple platform engineering.

Chapter 14

References

- [1] J. Deng et al., “ArcFace: Additive Angular Margin Loss for Deep Face Recognition,” CVPR, 2019.
- [2] Z. Yu et al., “Searching Central Difference Convolutional Networks for Face Anti-Spoofing,” CVPR, 2020.
- [3] InsightFace: 2D and 3D Face Analysis Project. <https://github.com/deepinsight/insightface>.
- [4] A. Huang, “Similarity Measures for Text Document Clustering,” NZCSRSC, 2008.
T. Soukupová and J. Čech, “Real-Time Eye Blink Detection using Facial Landmarks,” CVWW, 2016.
- [5] Apple Developer Documentation (Vision, Core ML, AVFoundation, LocalAuthentication, Keychain, NSWorkspace, SwiftUI, Accelerate). <https://developer.apple.com/documentation/>.
- [6] Apple Platform Security (Secure Enclave) & Machine Learning (Neural Engine). <https://support.apple.com/guide/security/>.
- [7] Y. Kolb, “XcodeGen,” <https://github.com/yonaskolb/XcodeGen>.
Apple Inc., “coremltools,” <https://github.com/apple/coremltools>.
SQLite, <https://www.sqlite.org/>.
- [8] A. Paszke et al., “PyTorch,” NeurIPS, 2019.
ONNX: Open Neural Network Exchange, <https://onnx.ai/>.